BOSCH

How Teams will be judged

Teams are ranked on a score out of **100.**

Part	Points
Checkpoint 1: Eyes	**10**
Checkpoint 2: Brain	**20**
Checkpoint 3: Hands	**20**
Checkpoint 4: Hidden test (unseen incidents, scored automatically)	**30**
Judges: safety and human approval, agent design and tool use, explainability, testing and honesty, demo	**20**



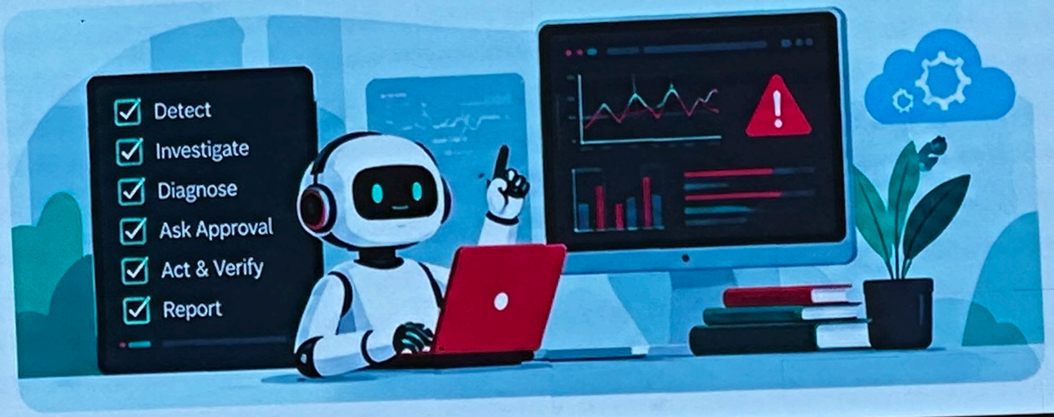
Individual assessment

Each student is also assessed individually through the checkpoints and a short one-to-one conversation.

BOSCH

Optional Stretch Goals

1. **Incident memory**
   Reuse earlier incident reports to diagnose a repeat incident faster.
2. **Reviewer agent**
   A second agent tries to disprove the diagnosis before the human sees it.
3. **Calibrated confidence**
   Show that the agent's stated confidence matches how often it is right on the practice set.
4. **Chat-style approval**
   The on-call human approves or rejects from a simple chat UI or dashboard.
5. **Prevention**
   After fixing, the agent suggests a code, config or alerting change that would stop the incident recurring.



Deliverables and Rules

Every team submits five items in its repository by the **16:30 code freeze**. There is no demo video; you demo live.

Deliverable	Details
Source code	Team repository forked from the template, with a README covering setup, how to run, how the approval gate and kill switch work, and known limitations.
Architecture overview	One page: components, the tools the agent uses, how it decides, and the safety controls (a photo of a whiteboard sketch is fine).
Incident reports	Reports produced by your agent for the practice incidents it handled, in the provided JSON schema plus readable form.
Evaluation note	Which practice incidents your agent got right and wrong in your own testing, and why.
Team form	Team name, members, who built which part, tools and AI models used (including AI coding assistants), third-party libraries.

Rules

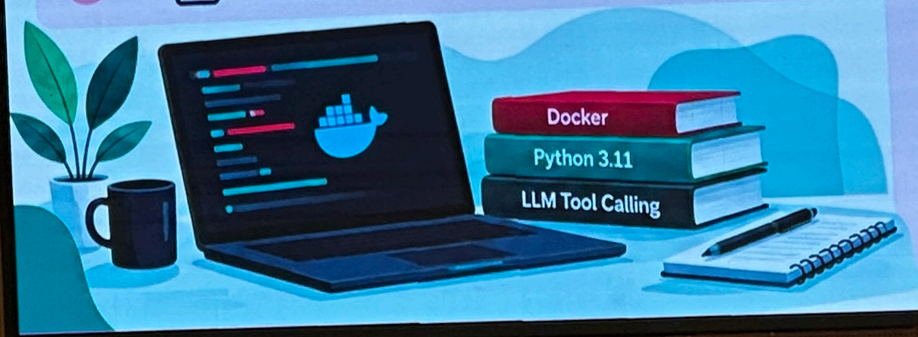
1. Teams of 5. All code is written on the day, from **09:15**; commits before then are not allowed.
2. Open-source libraries and agent frameworks are allowed. AI coding assistants are allowed if declared in the team form; every member must be able to explain and change the code they submit.
3. All three members must **commit code.**
4. The agent changes the demo **only through the Action API**, and **only after human approval.**
5. Mentors help you get unblocked; they **do not write code** for you. No outside help.
6. Reading feature-flag settings to diagnose, or tampering with the demo, faultctl, the hidden incidents or the scoring harness means **disqualification.**
7. Be honest about what does not work. Stating a limitation costs nothing, hiding a failure in the demo **loses points.**

BOSCH

Setup, Guidelines and Rules

Before the event
(from the prep pack)

1. Docker with Compose v2 and Python 3.11 installed.
2. The demo needs **about 6 GB of RAM and 14 GB** of disk; a 16 GB laptop is comfortable. On an 8 GB laptop, use the demo's minimal mode (about 3 GB, without Kafka and the flag UI) or the offline bundles.
3. Clone the demo and pull its images in advance (images also on USB drives on the day).
4. Run the demo once at home and open its feature-flag page, to see a failure end to end.
5. Read the 2-page primer on LLM tool calling.



On the day

1. Everything runs on **your own laptop;** nothing is deployed to shared infrastructure.
2. **Any language, agent framework** or open-source library is allowed.
3. Your agent's output for each incident must follow the **provided JSON schema**, so it can be scored.

Out of scope

1. Editing the demo's source code or configuration, faultctl or the scoring harness; the agent may act **only through the Action API.**
2. Reading the feature-flag settings (flagd files, flag UI or flag API) to diagnose an incident; **diagnosis must come from telemetry.**
3. Any action that runs **without human approval** (except read-only dry runs).
4. **Hard-coding answers** to the practice incidents.
5. Anything **outside your own laptop,** or that affects other teams or the venue network.
6. Attacking **real or production systems.**

BOSCH

Examples of Incidents Your Agent May Face

Application faults
(the demo's built-in failure flags)

- The payment service fails when charging
- The cart service fails intermittently
- The product catalog fails for one specific product
- A memory leak in the recommendation service from a runaway cache
- High CPU or heavy garbage collection in the ad service
- A traffic flood on the homepage

Infrastructure faults
(injected by organisers)

- A service container is stopped or keeps restarting
- The cache or a database becomes unavailable
- Network latency or packet loss between two services
- A service is starved of CPU or memory

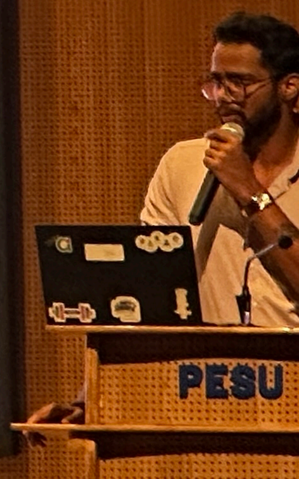


Ten practice incidents are published at kickoff. The final evaluation uses **unseen incidents**, including infrastructure faults and combinations, so build an agent that **reasons from telemetry**, not one that reads the answers from the feature-flag settings (that is not allowed, see Rules).

Starter kit and environment guidelines

Every team runs the same open-source application, so results are comparable and nobody loses the morning to setup. Organisers add a thin layer on top so the agent has safe, standard ways to look and act.

Provided	What it is	Source
OpenTelemetry Demo	Microservices store (frontend, cart, checkout, payment, product catalog, recommendation, ad, shipping and more) with a Locust load generator; run with Docker Compose	Open source GitHub
Telemetry	Traces, metrics and logs from every service through the OpenTelemetry Collector, with the bundled trace and metrics viewers	Comes with the demo
Failure flags	Built-in switches for realistic failures, managed by flagd	Comes with the demo
faultctl	Command-line tool to inject a practice incident (a flag or an infrastructure fault) and reset everything	Team
Action API	The only allowed way for your agent to change the system: restart a service, toggle a feature flag, throttle the load generator, scale a service (Kubernetes only). Every call is logged	Team
Offline incident bundles	Recorded telemetry for every practice incident, for laptops that cannot run the full demo	Team
LLM access	An API key with a spend limit; open-source or local models also allowed	Team
Template repository	Agent skeleton, required output format (JSON schema), README template, team form	Team



BOSCH

What a Team's Solution Should Do

1

Detect the incident

Receive an alert (an error-rate or latency threshold on the demo's metrics) and start an investigation automatically.

2

Investigate with tools

Give the agent tools to query traces, metrics, logs and container health. The agent should decide which tool to call next based on what it has found so far, rather than pasting all the data into one prompt.

Traces
Metrics
Logs
Container Health

3

Diagnose with evidence

Name the failing component and the cause, with a confidence level and the exact traces, log lines or metric values that support it. Evidence must exist in the data; invented evidence is penalised.

4

Propose a safe fix

Pick a remedy only from the allowlisted runbook actions in the Action API (restart a service, toggle a feature flag, throttle the load generator, scale a service on Kubernetes), with a short reason.

5

Ask for human approval

Show the diagnosis and plan to a human and wait. Nothing changes without an explicit yes. Provide a dry-run mode and a kill switch.

6

Act and verify

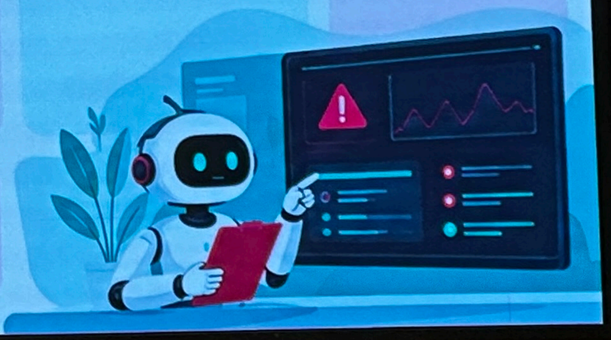
Execute the approved action, then check health metrics to confirm the system recovered. If it did not, roll back and escalate.

7

Report

Produce a short incident report: timeline, root cause, evidence, action taken, and what would prevent a repeat.

When the agent is **not confident, escalating to a human with its findings** is a correct outcome and **scores better** than a confident wrong fix.



BOSCH

Incident Zero: Build an AI On-Call Engineer

One-day AI hackathon at PES University

Student-facing brief

Theme

Agentic AI for IT Operations: from alert to safe, approved fix

Problem Statement

Build an **AI On-Call Engineer:** an agent that responds to a production incident in a cloud-native application, investigates it using real tools, identifies the root cause with evidence, and resolves it safely with human approval.

What does this mean?

When a production system breaks, an on-call engineer is paged, often at 2 a.m. They read logs, check dashboards, look at what was deployed recently, guess the cause, and apply a fix, all under pressure. Much of that first hour is searching and correlating, which is exactly what an AI agent can speed up.

Teams must build an AI agent that does this first hour of work. It receives an alert, decides which tools to call and in what order, reasons over logs, metrics, traces and deployment history, and names the root cause with the evidence behind it. It then proposes a fix from an approved runbook, asks a human for approval, applies it, and checks that the system has actually recovered.

A good agent is not the one that always acts. It is the one that acts when it has evidence, and says "I am not sure, here is what I found" when it does not.



Application for this Hackathon

OpenTelemetry Demo (Astronomy Shop)

Every team works on the same open-source, cloud-native application: the **OpenTelemetry Demo** (the "Astronomy Shop"), a microservices e-commerce store written in a dozen languages, running on your own laptop. It already sends traces, metrics and logs, and has built-in switches that cause realistic failures.

GitHub Repository

https://github.com/open-telemetry/opentelemetry-demo

PESU

BOSCH

Hackathon Flow

Two days. Build. Experiment. Learn. Showcase.



Day 1 — Build the foundation

11:30 – 12:00	12:00 – 13:00	13:00	13:00 – 15:30	15:30	15:30 – 18:30	18:30	18:45
Kickoff	**Setup**	**Checkpoint 0: Setup**	**The agent can see**	**Checkpoint 1: Eyes**	**The agent can diagnose**	**Checkpoint 2: Brain**	**Day 1 Wrap Up**
Problem reveal, starter kit walkthrough, rules	OpenTelemetry demo running with load; faultctl injects and resets an incident; LLM key works; repo forked; all 3 members have committed	Mentor confirms the setup; blockers are fixed on the spot	Tools for logs, metrics, traces and deploy history; agent summarises the symptoms of one practice incident; 1-page architecture sketch	Live run on one practice incident, plus the sketch	Root cause with evidence and confidence; output in the JSON schema	Correct diagnosis on at least 3 practice incidents chosen by the mentor	Day 1 code state saved; mentors review Day 1 progress

Day 2 — Enhance, analyze and showcase

09:00 – 09:30	09:30 – 12:00	12:00	12:00 – 13:30	13:30 – 14:00	14:00 – 15:30	15:30 – 16:30	16:30 – 18:45
Day 2 Alignment	**The agent can act safely**	**Checkpoint 3: Hands**	**Harden, document & prep submission**	**Final verification & submission**	**Checkpoint 4: Hidden test & mentor review**	**Evaluation & elimination round**	**Finalist demos & individual interviews**
Briefing for Day 2 goals, review feedback from Day 1 checkpoints	Allowlisted action proposed; approval gate; execute; recovery verified; dry run and kill switch; incident report generated	One full run, alert to verified recovery, with the action log	No new features; fix bugs; clean run from scratch; finish README, evaluation note and team form	Final code push, verify schema compliance and deliverables	Organisers run unseen incidents on your setup; automated scoring and reviewer evaluations	Mentors and judges review test scores and select qualifying teams for final demos	5-minute live demo plus 3 minutes of questions; short one-to-one technical conversation with shortlisted teams

PESU